Spring WebFlux

Классическая модель Spring MVC работает по принципу "один запрос - один поток". Входящий HTTP-запрос захватывает поток из пула, и этот поток занят на все время обработки, включая ожидание ответов от базы данных и внешних сервисов. Пока поток ждет I/O, он заблокирован и не может обслуживать другие запросы. Масштабирование в такой модели достигается за счет увеличения числа потоков или количества экземпляров приложения. Увеличение числа потоков имеет свои ограничения, так как платформенные потоки ресурс дорогой: каждый поток требует ~1 MB стека, а переключение контекста между ними создает нагрузку на планировщик ОС.

Spring WebFlux предлагает принципиально иной подход, построенный на неблокирующем I/O и модели event loop. Вместо того чтобы выделять поток на каждый запрос, WebFlux обслуживает тысячи одновременных соединений небольшим фиксированным набором потоков. Ключевая идея: поток никогда не блокируется в ожидании I/O, вместо этого он регистрирует интерес к событию (например, "данные прибыли на сокет") и переключается на обработку других готовых задач.

Netty как транспортный уровень

Spring WebFlux по умолчанию использует Reactor Netty - обертку над Netty, предоставляющую Reactive Streams API поверх асинхронного фреймворка. При запуске Spring Boot приложения с зависимостью `spring-boot-starter-webflux` происходит следующая цепочка инициализации:

1. `NettyReactiveWebServerFactory` создает экземпляр `reactor.netty.http.server.HttpServer` .
2. `HttpServer` конфигурируется через `ReactorResourceFactory` , которая управляет общими ресурсами - event loop потоками (`LoopResources`) и пулом соединений (`ConnectionProvider`).
3. Создается `NettyWebServer` (`org.springframework.boot.reactor.netty.NettyWebServer`) - реализация интерфейса Spring Boot `WebServer` , которая оборачивает `HttpServer` и `ReactorHttpHandlerAdapter` (мост между Spring WebFlux `HttpHandler` и Reactor Netty).
4. При вызове `NettyWebServer.start()` сервер привязывается к порту и начинает принимать соединения.

EventLoopGroup и EventLoop

Центральная абстракция Netty - `io.netty.channel.EventLoopGroup` , контейнер для объектов `EventLoop` .
Каждый `EventLoop` эксклюзивно привязан к одному платформенному потоку. Количество event loop потоков определяется интерфейсом `reactor.netty.resources.LoopResources` , который по умолчанию использует формулу:

```
int DEFAULT_IO_WORKER_COUNT = Integer.parseInt(System.getProperty(
        ReactorNetty.IO_WORKER_COUNT,
        "" + Math.max(Runtime.getRuntime().availableProcessors(), 4)));
```

Объяснить с SourceCraft

То есть количество потоков равно числу доступных процессоров, но не менее 4. Это значение можно переопределить:

- JVM-параметром `-XX:ActiveProcessorCount=N` - влияет на `Runtime.getRuntime().availableProcessors()`
- системным свойством `-Dreactor.netty.ioWorkerCount=N` - прямое задание количества event loop потоков.

В контейнерной среде (Docker, Kubernetes) JVM определяет число доступных процессоров через cgroups. Если контейнеру выделено 2 CPU - `availableProcessors()` вернет 2, но поскольку минимум равен 4, будет создано 4 event loop потока. Это поведение можно скорректировать явным указанием `-Dreactor.netty.ioWorkerCount=2` для соответствия фактическим ресурсам контейнера.

I/O - мультиплексирование

В основе всей модели лежит способность операционной системы эффективно отслеживать состояние тысяч файловых дескрипторов одним системным вызовом, другими словами, выполнять I/O мультиплексирование. Netty динамически определяет стратегию I/O-мультиплексирования в зависимости от операционной системы, на которой запускается приложение (если в classpath есть соответствующие модули):

- `EpollIoHandler` - на Linux, модуль `netty-transport-native-epoll`. Использует нативные системные вызовы `epoll_create`, `epoll_ctl`, `epoll_wait` напрямую через JNI.
- `KQueueIoHandler` - на macOS, модуль `netty-transport-native-kqueue`. Использует системные вызовы `kqueue` / `kevent`.
- `NioIoHandler` - fallback-реализация через стандартный Java NIO. На Linux NIO `Selector` внутри тоже использует epoll, но через прослойку Java NIO, что добавляет небольшой overhead.

Как правило, при создании Docker-образов приложения в качестве базового образа используется тот или иной Linux, поэтому в большинстве случаев мы будем работать именно с `EpollIoHandler`. Подробнее про механизм `epoll` можно почитать в официальной документации. Именно он позволяет зарегистрировать событие "интереса" на файловом дескрипторе и получить уведомление, когда дескриптор будет готов к работе. Например, поток может зарегистрировать событие ожидания готовности сокета к чтению данных (`EPOLLIN`) через системный вызов epoll_ctl , после этого в цикле epoll_wait получать уведомления обо всех событиях, которые он ранее зарегистрировал, и, когда для нужного сокета придет уведомление, поток будет знать, что данные гарантированно есть, и можно читать без ожидания.

Channel и Channel affinity

Каждое сетевое соединение в Netty представлено объектом `io.netty.channel.Channel`. Для TCP-соединений используются конкретные реализации: `NioSocketChannel`, `EpollSocketChannel` или `KQueueSocketChannel` в зависимости от выбранного транспорта.

При приеме нового TCP-соединения серверный `Channel` (`NioServerSocketChannel` / `EpollServerSocketChannel`) вызывает `EventLoopGroup.next()`, который по своему алгоритму выбирает `EventLoop` и регистрирует на нем новый клиентский `Channel`. С этого момента канал навсегда привязан конкретному event loop - все операции с ним (чтение, запись, обработка событий) всегда выполняются в одном и том же потоке. Это фундаментальное свойство называется channel affinity.

Обратная связь при этом асимметрична: один event loop может обслуживать сотни и тысячи каналов, но каждый канал закреплен ровно за одним event loop.

Channel affinity дает два важных преимущества:

1. Отсутствие синхронизации - все операции с каналом гарантированно выполняются в одном потоке, поэтому не требуются блокировки, CAS-операции или volatile-переменные при работе с состоянием конкретного соединения.
2. Эффективность кэша процессора - данные конкретного соединения (буферы, состояние `ChannelPipeline`) остаются в L1/L2 кэше одного ядра, не мигрируя между ядрами при каждой операции (если не было переключения контекста из-за слишком большого количества `EventLoop` потоков).

Обработка запроса в ChannelPipeline

Каждый `Channel` в Netty содержит `ChannelPipeline` - упорядоченную цепочку обработчиков(`ChannelHandler`), через которую проходят входящие и исходящие события. Для HTTP-сервера Reactor Netty конфигурирует pipeline следующими обработчиками:

1. `io.netty.handler.codec.http.HttpServerCodec` - комбинированный кодек, который декодирует входящие байты в объекты `HttpRequest` и кодирует исходящие `HttpResponse` в байты.
2. `reactor.netty.http.server.HttpTrafficHandler` - управляет жизненным циклом HTTP-соединения: keep-alive, pipelining, обработка `100-continue` и т.д.
3. `reactor.netty.http.server.HttpServerOperations` - мост между Netty и Reactor. Реализует интерфейсы `HttpServerRequest` / `HttpServerResponse` из Reactor Netty API и связывает Netty-канал с реактивными потоками Reactor.

Далее `ReactorHttpHandlerAdapter` передает запрос в `DispatcherHandler` Spring WebFlux, который маршрутизирует его к соответствующему контроллеру. Весь pipeline выполняется в потоке event loop, к которому привязан канал.

Из описанной модели следует главное ограничение: любая блокирующая операция в event loop потоке останавливает обработку всех каналов, привязанных к этому event loop. Если у event loop 500 активных соединений и обработчик одного из них вызвал `Thread.sleep(100)` или блокирующий JDBC-запрос - все 500 соединений не получают обслуживания в течение всего времени блокировки.

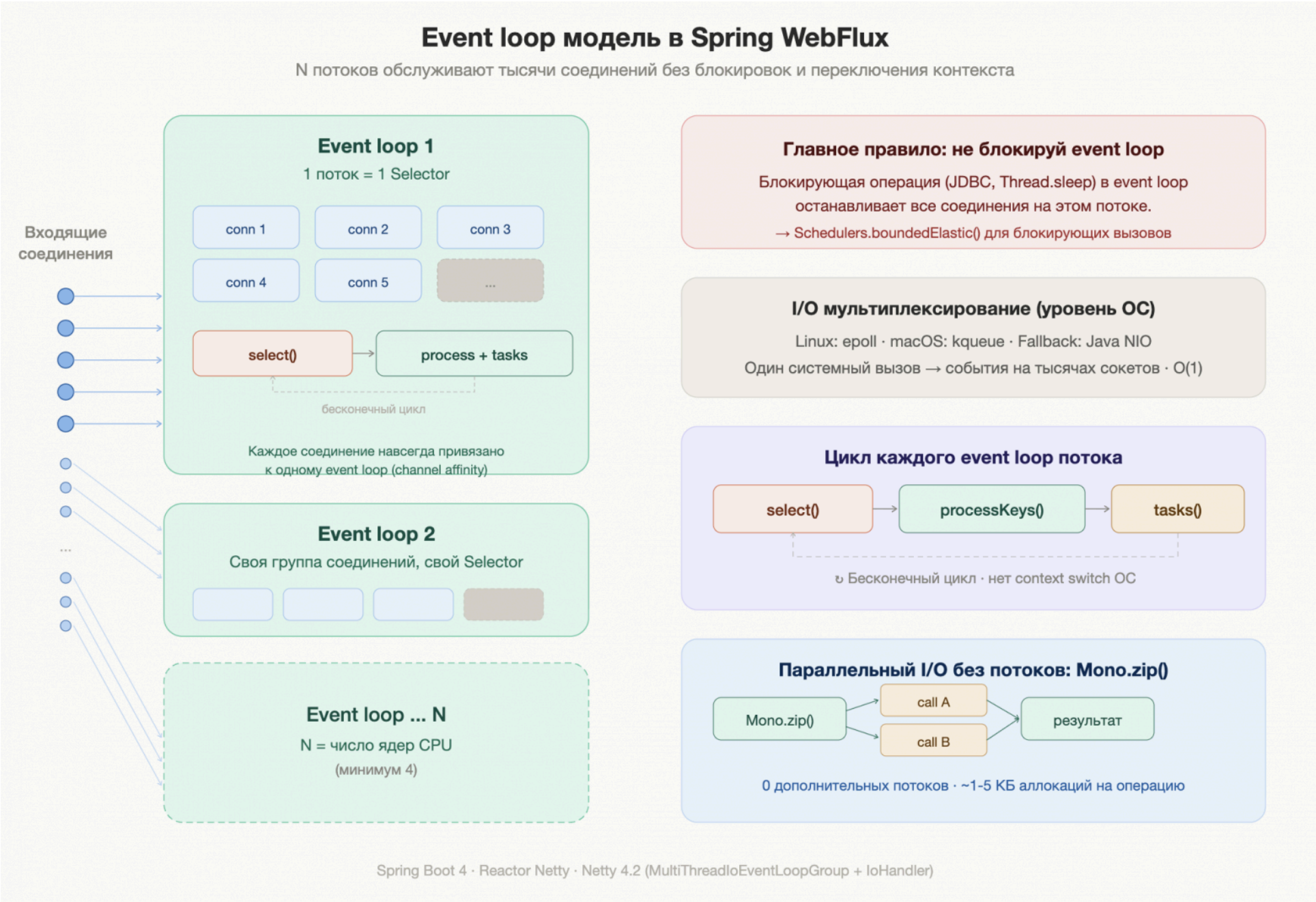
Это не баг, а архитектурное следствие. Модель event loop достигает высокой производительности именно потому, что потоки никогда не блокируются - весь I/O асинхронный, а бизнес-логика выражена через реактивные цепочки, которые не содержат точек блокировки.

Если же в приложении есть блокирующие операции (JDBC, блокирующий HTTP-клиент) или CPU-интенсивные вычисления, их необходимо выносить на отдельный пул потоков, например, через оператор `.subscribeOn(Schedulers.boundedElastic())`.

Ограничения модели

При всех преимуществах, модель event loop имеет существенные ограничения:

1. Реактивный код на Reactor (цепочки `map`, `flatMap`, `zip`, `switchIfEmpty`) значительно сложнее для написания и отладки, чем императивный код. Стектрейсы в реактивных приложениях часто нечитаемы - в стеке видны внутренние классы Reactor (`FluxMapFuseable`, `MonoFlatMap`), а не бизнес-логика. Для улучшения диагностики можно использовать `Hooks.onOperatorDebug()`, но это добавляет заметный overhead.
2. Экосистема блокирующих библиотек (JDBC, многие SDK) несовместима с моделью event loop. Для работы с реляционными СУБД необходим R2DBC - реактивный драйвер, набор поддерживаемых СУБД у которого уже, чем у JDBC. Альтернативный подход - вынос блокирующих вызовов на `Schedulers.boundedElastic()` или `Executors.newVirtualThreadPerTaskExecutor()`.
3. Стандартные инструменты профилирования менее информативны для реактивных приложений: thread dump показывает N event loop потоков, каждый из которых стоит в вызове `epoll_wait()` / `kevent()` / `Selector.select()`, что не дает информации о том, какие именно запросы обрабатываются в данный момент.
4. Одна "тяжелая" операция в event loop (CPU-интенсивный расчет, случайно попавший блокирующий вызов) деградирует не один запрос, а все запросы на этом event loop. При 4 event loop потоках - это потенциально четверть всего трафика приложения.



Event Loop модель

Virtual Threads

Виртуальные потоки (Project Loom, JEP 444) не меняют модель "один запрос - один поток", они меняют стоимость потока. Каждый запрос по-прежнему получает свой поток, код остается императивным и синхронным, стектрейсы читаемы, `ThreadLocal` работает. Но теперь "поток" - это легкая Java-конструкция, стоимость которой сравнима с обычным объектом в heap.

Carrier-потоки и планировщик

Виртуальные потоки не исполняются сами по себе - JVM "монтирует" их на платформенные потоки, называемые carrier-потоками. Эти потоки живут в специальном `ForkJoinPool`, работающем в режиме FIFO, он создается один раз при старте JVM

и используется всеми виртуальными потоками.

Количество carrier-потоков определяется свойством `jdk.virtualThreadScheduler.parallelism` и по умолчанию равно числу доступных процессоров (`Runtime.getRuntime().availableProcessors()`). В контейнере с 2 CPU это означает всего 2 carrier-потока - и именно на этих 2 платформенных потоках могут поочередно исполняться тысячи виртуальных.

Метод создания планировщика по умолчанию в классе `VirtualThread.java` .

```
private static ForkJoinPool createDefaultScheduler() {
    ForkJoinWorkerThreadFactory factory = pool -> new CarrierThread(pool);
    int parallelism, maxPoolSize, minRunnable;
    String parallelismValue = System.getProperty("jdk.virtualThreadScheduler.parallelism");
    String maxPoolSizeValue = System.getProperty("jdk.virtualThreadScheduler.maxPoolSize");
    String minRunnableValue = System.getProperty("jdk.virtualThreadScheduler.minRunnable");
    if (parallelismValue != null) {
        parallelism = Integer.parseInt(parallelismValue);
    } else {
        parallelism = Runtime.getRuntime().availableProcessors();
    }
    if (maxPoolSizeValue != null) {
        maxPoolSize = Integer.parseInt(maxPoolSizeValue);
        parallelism = Integer.min(parallelism, maxPoolSize);
    } else {
        maxPoolSize = Integer.max(parallelism, 256);
    }
    if (minRunnableValue != null) {
        minRunnable = Integer.parseInt(minRunnableValue);
    } else {
        minRunnable = Integer.max(parallelism / 2, 1);
    }
    Thread.UncaughtExceptionHandler handler = (t, e) -> { };
    boolean asyncMode = true; // FIFO
    return new ForkJoinPool(parallelism, factory, handler, asyncMode,
                 0, maxPoolSize, minRunnable, pool -> true, 30, SECONDS);
}
```

Объяснить с

Как это возможно? Благодаря механизму mount/unmount.

Когда виртуальный поток готов к работе, планировщик монтирует его на свободный carrier-поток: восстанавливает стек вызовов из heap и передает управление. С этого момента carrier-поток исполняет код виртуального потока - проходит через фильтры, контроллер, сервисный слой - точно так же, как обычный платформенный поток.

Самое интересное происходит, когда код доходит до блокирующей операции, скажем, `repository.findById(id)` , и ждет ответа от БД. В этот момент JVM выполняет unmount:

1. Стек вызовов виртуального потока сохраняется в объекты `StackChunk` в Java heap.
2. Виртуальный поток переходит в состояние `PARKED` .
3. Carrier-поток освобождается и немедленно подхватывает из рабочей очереди ForkJoinPool следующий виртуальный поток, готовый к исполнению.

С точки зрения бизнес-кода ничего не произошло - `repository.findById(id)` просто еще не вернул результат. Но carrier-поток уже обслуживает другой запрос.

Когда данные от БД прибудут, виртуальный поток будет "разбужен" и поставлен обратно в очередь планировщика. При следующей возможности (возможно, на другом carrier-потоке) стек восстановится из heap, и исполнение продолжится ровно с того места, где остановилось.

За время обработки одного HTTP-запроса, включающего вызов внешнего сервиса и запрос к БД, виртуальный поток может пройти через несколько циклов mount/unmount, каждый раз освобождая carrier для другой работы.

JVM Poller - под капотом виртуальных потоков

Остается вопрос: кто "будит" виртуальный поток, когда данные на сокете готовы?

Этим занимается Poller - механизм, подробно описанный в статье Networking I/O with Virtual Threads — Under the hood. Он работает следующим образом: когда виртуальный поток делает блокирующий вызов, скажем, `socket.read()` , и данные еще не готовы, JVM переводит нативный сокет в неблокирующий режим и регистрирует его файловый дескриптор в Poller'е. Стоит уточнить, что сокет переводится в неблокирующий режим один раз, при первом вызове из виртуального потока, а не каждый раз.

JVM создает два выделенных платформенных потока:

- Read-Poller - отслеживает готовность к чтению, connect и accept.
- Write-Poller - отслеживает готовность к записи.

Каждый из них крутит цикл с вызовом I/O-мультиплексора ОС - того же самого, что использует Netty:

- На Linux - EPollPoller, вызывающий `epoll_wait()` .
- На macOS - KQueuePoller, вызывающий `kevent()` .

Когда ОС сообщает, что на сокете появились данные, Poller вызывает `VirtualThread.unpark()` - виртуальный поток ставится в очередь `ForkJoinPool` и при следующей возможности монтируется на carrier-поток.

По сути, виртуальные потоки и WebFlux используют один и тот же механизм ОС (`epoll` / `kqueue`) для отслеживания готовности I/O, но на разных уровнях абстракции. Netty работает с `epoll` напрямую в event-loop потоке, а виртуальные потоки - через промежуточный слой Poller.

Нюансы настройки

Параметр `parallelism` , пожалуй, один из наиболее важных для тюнинга работы сервисов на виртуальных потоках в Docker/Kubernetes. По умолчанию он равен числу CPU из cgroups. Для контейнера с 2 CPU - `parallelism=2` , то есть всего 2 carrier-потока. Казалось бы, для I/O-bound приложения 2 потока достаточно: виртуальные потоки ведь размонтируются при I/O. Но между блокирующими вызовами есть участки CPU-работы: парсинг JSON, маппинг объектов, валидация, и 2 carrier-потока становятся узким местом. Дополнительные потоки обеспечивают более быстрый подхват виртуальных потоков из очереди. Но при значительном превышении числа ядер overhead от переключений контекста планировщиком ОС начинает доминировать.

Также стоит учесть, что не все блокирующие операции приводят к размонтированию виртуального потока. Есть два особых случая, о которых важно знать.

- Компенсация. Некоторые операции блокируют carrier-поток, но JDK знает об этом и создает дополнительный компенсационный carrier-поток для поддержания параллелизма. Примеры: файловый I/O, `Object.wait()` . Максимальное число таких компенсационных потоков ограничено параметром `jdk.virtualThreadScheduler.maxPoolSize` (по умолчанию 256). Компенсационные потоки живут 30 секунд после создания, даже если больше не нужны.
- Pinning. Виртуальный поток "прикреплен" к carrier-потоку и не может размонтироваться, например, при исполнении нативного кода или foreign functions. До JDK 24 аналогичная проблема существовала для `synchronized` -блоков (JEP 491 устранил это). Критически важно: планировщик не компенсирует pinning. Если виртуальный поток "прикрепился" (pinned) к carrier-потоку, фактический параллелизм снижается, и все остальные виртуальные потоки ждут в очереди.

Потребление ресурсов

Виртуальные потоки дешевле платформенных, но не бесплатны. Два аспекта, за которыми стоит следить:

- Стек каждого виртуального потока хранится в объектах `StackChunk` в Java heap. При глубоком стеке (а стек Spring MVC запроса проходит через десятки фреймов: Tomcat, фильтры, Spring, Hibernate, JDBC) каждый виртуальный поток может потреблять десятки килобайт heap. При тысячах одновременных VT это создает ощутимое давление на GC. Стоит уточнить: это оценка для парковки, когда стек сохранен в `StackChunk` . Пока виртуальный поток смонтирован на carrier, его стек находится на нативном стеке carrier-потока и отдельного потребления heap не создает.
- Каждый виртуальный поток имеет собственный `ThreadLocalMap` . При тысячах VT объем ThreadLocal-данных может быть значительным, особенно с учетом Spring Security context, MDC, Micrometer observation context.

Включение в Spring Boot

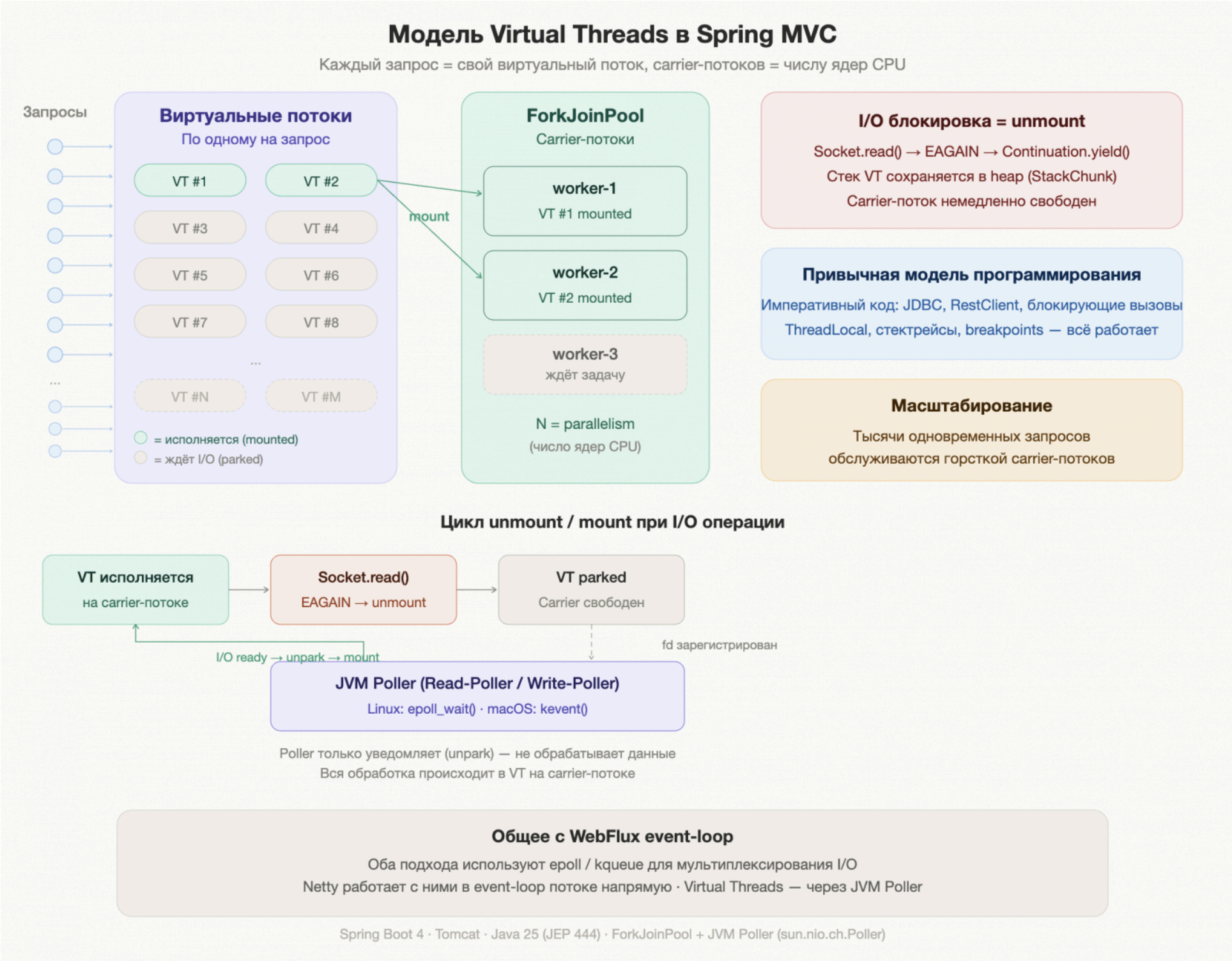
Начиная с Spring Boot 3.2, достаточно одной строки:

```yaml
spring:
  threads:
    virtual:
      enabled: true
```

Объяснить с

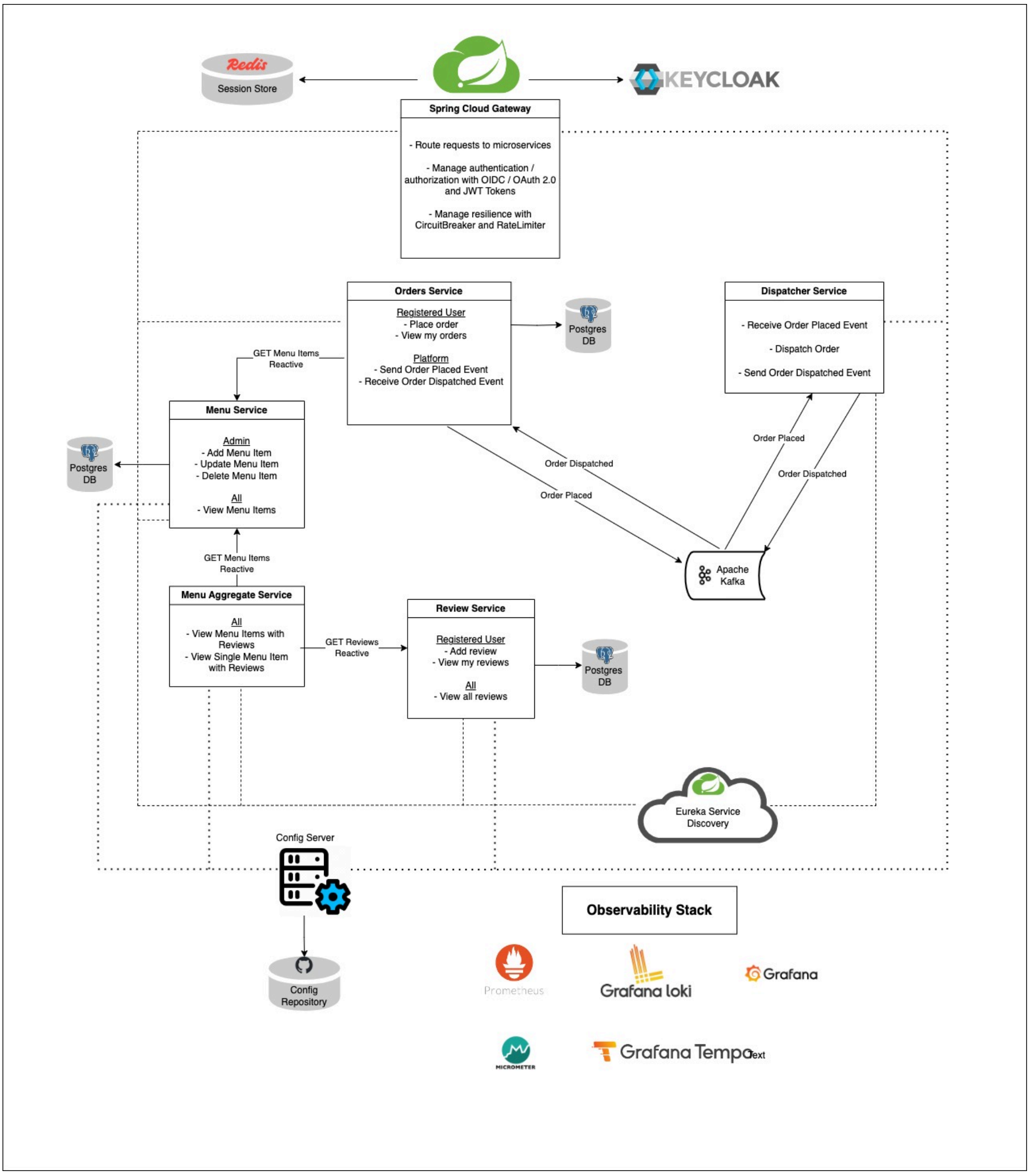
В этом случае Spring Boot активирует `TomcatVirtualThreadsWebServerFactoryCustomizer`, который настраивает `ProtocolHandler` Tomcat-а на использование `VirtualThreadExecutor`. Вместо извлечения потока из фиксированного пула Tomcat теперь создает новый виртуальный поток на каждый входящий запрос. Соответственно, все настройки размера пула потоков Tomcat уже не будут действовать. Помимо обработки HTTP-запросов, это свойство переключает на виртуальные потоки `@Async`-методы и `TaskScheduler`.

Более подробно о работе с виртуальными потоками и нюансами профилирования можно почитать в статье Java's Concurrency Revolution: How Immutability and Virtual Threads Changed Everything!



Модель Virtual Threads

Архитектура тестируемого приложения



Архитектура тестируемого приложения

Не вдаваясь в детали, верхнеуровневая архитектура такая - это бэкенд для ресторана с возможностью заказа еды навынос, состоящий из:

- пяти бизнесовых микросервисов
- нескольких БД PostgreSQL
- Redis для хранения информации о рейт лимитах
- входной точки в приложение в виде Spring Cloud Gateway Server (Reactive)
- Netflix Eureka Service Discovery
- Spring Cloud Config Server
- Сервера аутентификации и авторизации Keycloak

- Apache Kafka + Kafka Connect + Debezium Postgres Connector
- Инструментов Monitoring и Observability: Grafana, Loki, Tempo, Prometheus

Используется Spring Boot 4 и Java 25. Из пяти основных сервисов два изначально были реализованы на Spring WebFlux:

- Orders Service - сервис управления заказами
- Menu Aggregate Service - сервис, предоставляющий API для получения агрегированной информации о блюдах, их рейтингах и отзывах.

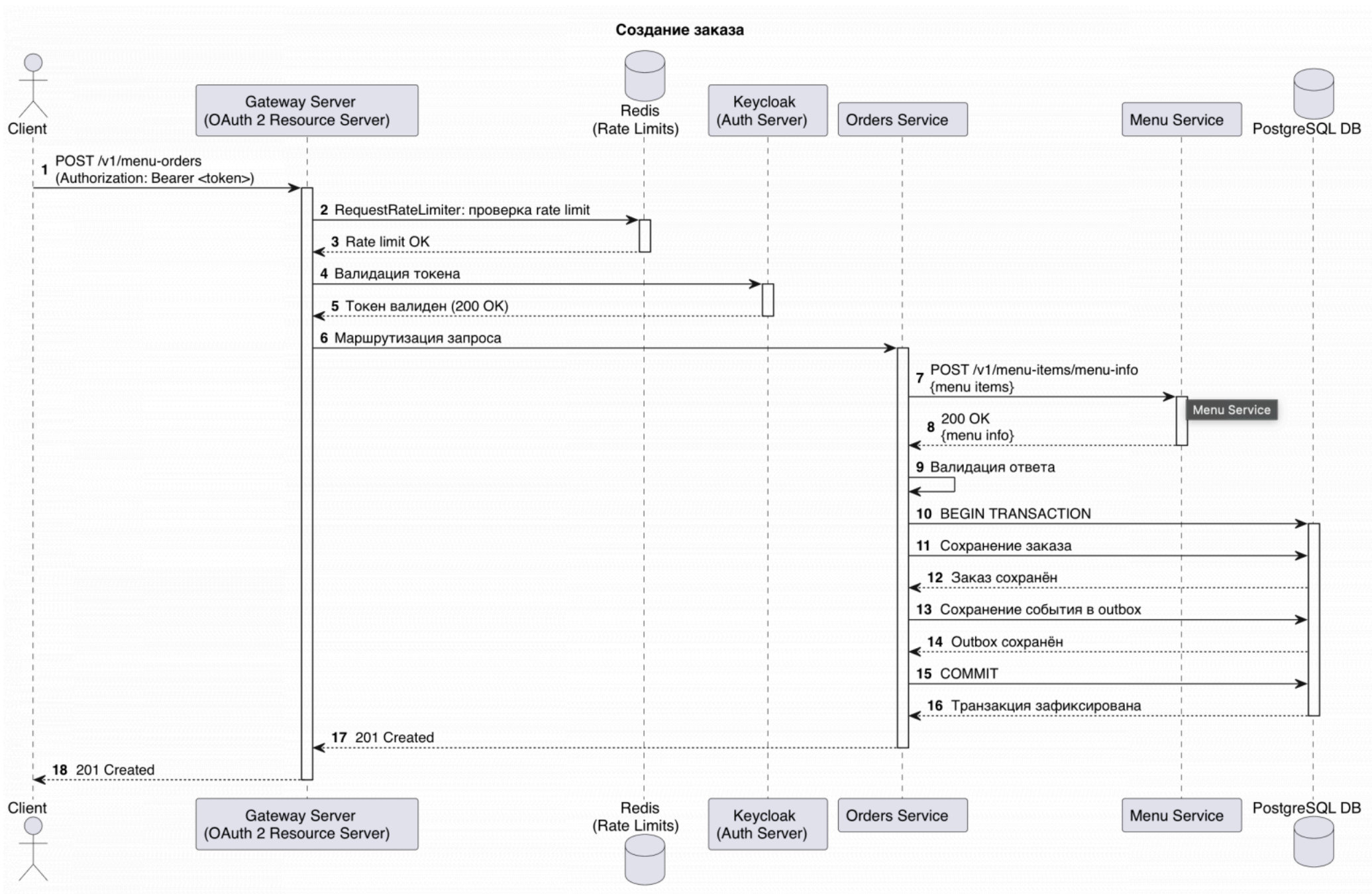
Я перевел эти сервисы на виртуальные потоки и решил сравнить поведение под нагрузкой, а результатами эксперимента поделиться с общественностью. Надеюсь, будет полезно. Итак, приступим.

Исходные данные эксперимента

- Тестирование проводилось на MacBook Pro с процессором M4 Pro, 48 Gb RAM, 14 CPU.
- Все микросервисы и инфраструктура разворачивались локально в Docker через Docker Compose.
- Каждому бизнесовому микросервису выделялось по 2 Gb памяти и 2 CPU.

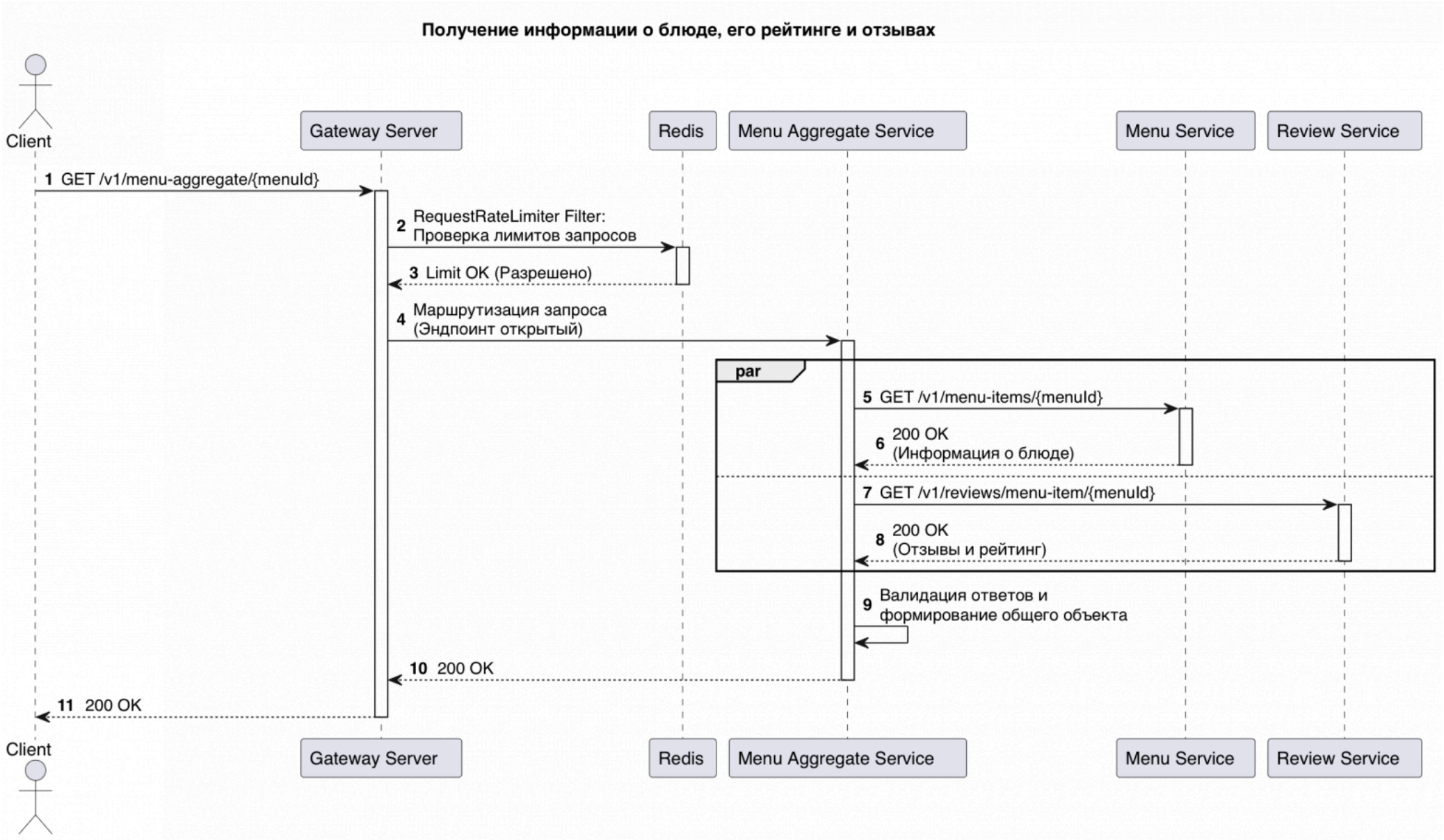
Тестировались следующие эндпоинты (представлен упрощенный вариант без проверок на ошибки):

- Создание заказа



Создание заказа

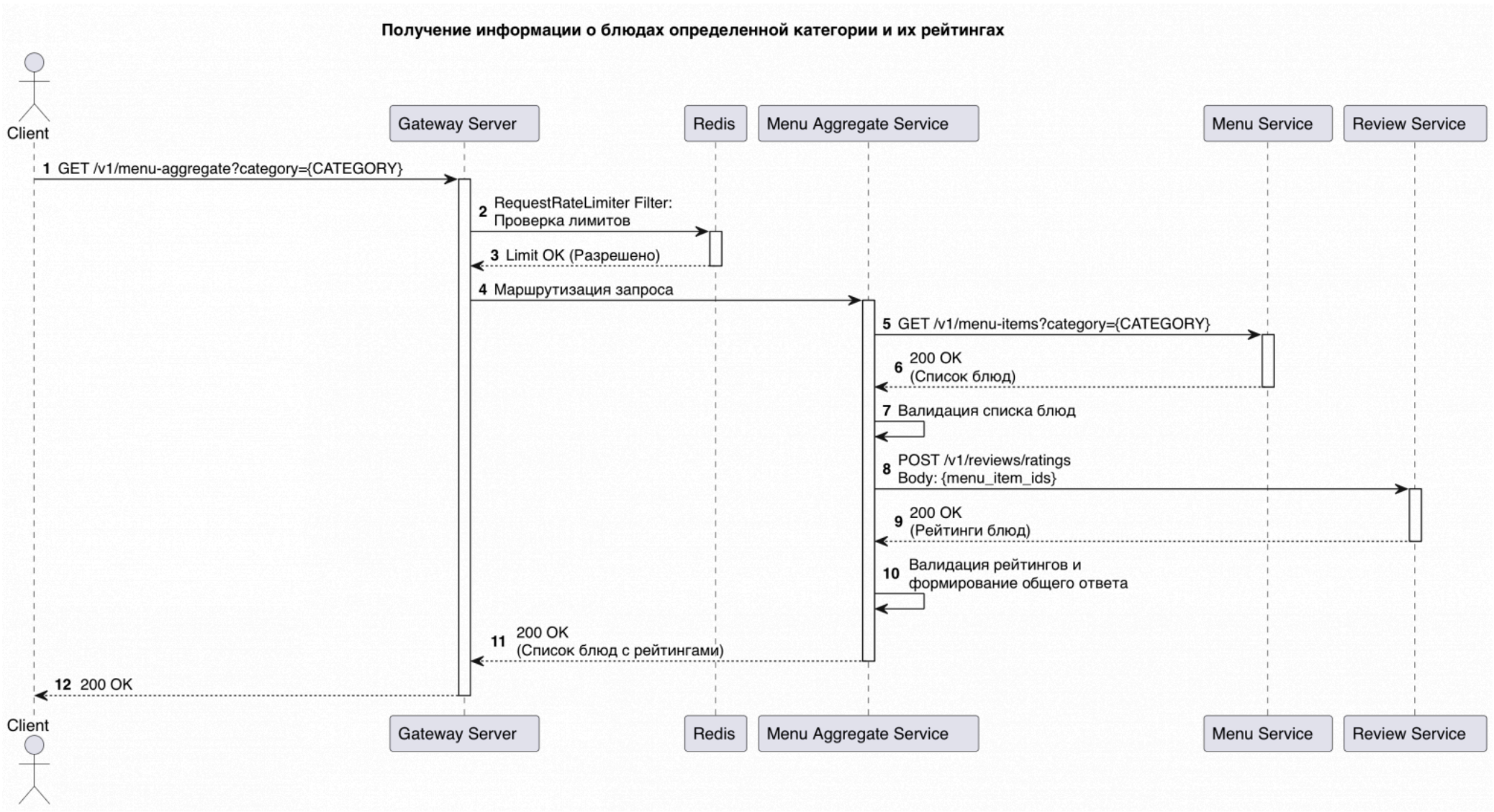
- Получение информации о блюде, его рейтинге и отзывах



Информация о блюде

В конфигурации с виртуальными потоками параллельные вызовы осуществлялись с помощью экспериментальной технологии StructuredConcurrency (JEP 505), на Spring WebFlux использовался `Mono.zip()` .

- Получение информации о блюдах определенной категории и их рейтингах



Список блюд

Генератор нагрузки - Gatling. Сценарий подачи нагрузки на все эндпоинты одинаковый:

1. В течение 10 секунд поток 2rps.
2. Далее в течение 5 секунд рост нагрузки до 100rps.
3. Постоянная нагрузка в 100rps в течение 30 секунд.

4. Рост до 200rps в течение 5 секунд.
5. Постоянная нагрузка в 200rps в течение 30 секунд.
6. Резкий рост нагрузки до 2000rps в течение 10 секунд.
7. Постоянная нагрузка в 2000rps в течение 10 минут.

Переменные параметры:

- Сервисы на Spring WebFlux тестировались с различными параметрами `-XX:ActiveProcessorCount` (4, 6, 8, 16)
- Сервисы на виртуальных потоках тестировались с параметрами:
  - `-Djdk.virtualThreadScheduler.parallelism` (2, 4, 6, 8, 16)
  - Для настройки `RestClient` использовались различные `ClientHttpRequestFactory` :
    - `HttpComponentsClientHttpRequestFactory` в двух конфигурациях:
      - С параметрами по умолчанию `DEFAULT_MAX_TOTAL_CONNECTIONS = 25` и `DEFAULT_MAX_CONNECTIONS_PER_ROUTE = 5` .
      - С параметрами `maxTotalConnections = 200` и `maxConnectionsPerRoute = 200` (тестирование проводилось только в связке с `-Djdk.virtualThreadScheduler.parallelism = 8`).
    - `JdkClientHttpRequestFactory` с Executor `Executors.newVirtualThreadPerTaskExecutor()` - если не указать свой Executor, то используется `Executors.newCachedThreadPool()` , что приводит к катастрофическому потреблению памяти при нагрузке в 2000rps.

Сразу оговорюсь, `JdkClientHttpRequestFactory` тестировалась не на всех эндпоинтах, так как не рассматривалась в качестве основной фабрики. `HttpComponentsClientHttpRequestFactory` имеет гораздо больше настраиваемых параметров и, на мой взгляд, более пригодна для промышленной эксплуатации несмотря на то, что в ряде случаев при использовании `JdkClientHttpRequestFactory` времена ответов были лучше. Также в некоторых случаях прогоны проводились не со всеми параметрами `-XX:ActiveProcessorCount` или `-Djdk.virtualThreadScheduler.parallelism` , так как по результатам предыдущих прогонов было очевидно, что какие-то параметры приводят к деградации по времени ответа, потреблению CPU или памяти.

Дальше будут цифры, ради которых проводился эксперимент.

Сводная сравнительная таблица

GET /v1/menu-aggregate/{id} — Menu Aggregate Service

Эксперимент	p95 (ms)	p99 (ms)	Heap	Non-Heap	CPU	Alloc Rate	GC avg (ops/s)	GC pause avg
VT, HttpComponents, parallelism=2	24	203	608 MiB	121 MiB	54%	450 MB/s	2.4	3 ms
VT, HttpComponents, parallelism=4	18	116	320 MiB	118 MiB	68%	450 MB/s	2.9	5 ms
VT, HttpComponents, parallelism=6	15	47	300 MiB	121 MiB	73%	450 MB/s	2.9	6 ms
VT, HttpComponents, parallelism=8	14	39	200 MiB	112 MiB	76%	450 MB/s	3.8	3 ms
VT, HttpComponents, parallelism=16	16	61	290 MiB	121 MiB	80.8%	450 MB/s	3.5	4 ms
VT, HttpComponents, parallelism=8, maxConn=200	13	31	273 MiB	120 MiB	73.3%	450 MB/s	4.3	3 ms
WebFlux, APC=4	13	29	207 MiB	125 MiB	70%	500 MB/s	4.0	3 ms
WebFlux, APC=6	10	24	204 MiB	126 MiB	71.3%	500 MB/s	4.1	3 ms

WebFlux, APC=8	9	23	204 MiB	128 MiB	73.8%	500 MB/s	4.0	3 ms
WebFlux, APC=16	11	31	214 MiB	129 MiB	69.5%	500 MB/s	4.0	4 ms

Оптимальная конфигурация Virtual Threads

Лучшие показатели у прогона с фабрикой `HttpComponentsClientHttpRequestFactory` при `-Djdk.virtualThreadScheduler.parallelism=8`, `maxConnectionsPerRoute=200` и `maxConnectionsTotal=200`:

- p95=13ms
- p99=31ms
- 273 Mb heap
- 73.3% CPU

Оптимальная конфигурация WebFlux

Лучший показатели у прогона с `ActiveProcessorCount=8`:

- p95=9ms
- p99=23ms
- 204 Mb heap
- 73.8% CPU

Конфигурация с `ActiveProcessorCount=6` показала практически идентичные результаты (p95=10 ms, p99=24 ms) с несколько меньшим потреблением CPU (71.3%).

Сравнение подходов

Для данного эндпоинта WebFlux демонстрирует заметное преимущество. Лучшая конфигурация WebFlux опережает лучшую конфигурацию Virtual Threads:

- p99 на 8 ms (23 vs 31 ms) ~ 34%
- p95 на 4 ms (9 vs 13 ms) ~ 44%
- при этом потребляя на 25% меньше heap-а (204 vs 273 Mb)
- потребление CPU примерно одинаковое

Причина кроется в природе эндпоинта: два параллельных I/O-вызова - это именно тот паттерн, где event-loop модель Netty наиболее эффективна. StructuredConcurrency на виртуальных потоках вносит дополнительные накладные расходы на создание и координацию fork/join.

GET /v1/menu-aggregate — Menu Aggregate Service

Эксперимент	p95 (ms)	p99 (ms)	Heap	Non-Heap	CPU	Alloc Rate	GC avg (ops/s)	GC pause avg
VT, HttpComponents, parallelism=2	42	126	310 MiB	120 MiB	69.3%	450 MB/s	3.0	3 ms
VT, HttpComponents, parallelism=4	27	151	420 MiB	119 MiB	72.3%	450 MB/s	2.5	4 ms
VT, HttpComponents, parallelism=8	24	89	264 MiB	122 MiB	81.5%	450 MB/s	3.5	3 ms
VT, HttpComponents, parallelism=16	34	124	328 MiB	122 MiB	84.5%	450 MB/s	2.3	3 ms

VT, HttpComponents, parallelism=8, maxConn=200	14	32	273 MiB	121 MiB	72.8%	450 MB/s	3.1	3 ms
VT, JdkClient, parallelism=2	57	274	554 MiB	122 MiB	85.3%	620 MB/s	2.3	4 ms
VT, JdkClient, parallelism=4	28	84	652 MiB	122 MiB	90.5%	620 MB/s	3.2	4 ms
VT, JdkClient, parallelism=6	19	41	447 MiB	122 MiB	95.5%	620 MB/s	3.4	4 ms
VT, JdkClient, parallelism=8	36	124	652 MiB	122 MiB	95.5%	620 MB/s	3.0	4 ms
WebFlux, APC=4	11	23	195 MiB	127 MiB	66%	580 MB/s	4.4	2 ms
WebFlux, APC=6	11	37	191 MiB	128 MiB	74.8%	580 MB/s	4.4	1.5 ms
WebFlux, APC=8	12	27	214 MiB	128 MiB	76%	580 MB/s	4.2	2.5 ms
WebFlux, APC=16	12	29	210 MiB	129 MiB	78%	580 MB/s	4.1	2 ms

Оптимальная конфигурация Virtual Threads

Безусловный лидер - `HttpComponentsClientHttpRequestFactory` при - `Djdk.virtualThreadScheduler.parallelism=8` , `maxConnectionsPerRoute=200` и `maxConnectionsTotal=200` :

- p95=14ms
- p99=32ms
- 273 MiB heap
- 72.8% CPU

Конфигурации на `JdkClientHttpRequestFactory` показали себя хуже по всем метрикам:

- значительно выше heap (447–652 MiB)
- выше CPU (85–95.5%)
- выше allocation rate (620 vs 450 MB/s).

Лучший прогон `JdkClientHttpRequestFactory` при `-Djdk.virtualThreadScheduler.parallelism=8` достиг p95=19 ms, p99=41 ms, но при этом потреблял 447 MiB heap и 95.5% CPU.

Оптимальная конфигурация WebFlux

Лучший результат - `ActiveProcessorCount=4` :

- p95=11ms
- p99=23ms
- 195 MiB heap
- 66% CPU

Все конфигурации WebFlux показали стабильно хорошие времена (p95 11–12ms, p99 23–37ms), минимальное потребление памяти (191–214 MiB) и предсказуемый GC (4.1–4.4 ops/s, паузы 1.5–2.5 ms).

Сравнение подходов

WebFlux снова впереди, хотя разрыв меньше, чем для первого эндпоинта. Лучший WebFlux опережает лучший Virtual Threads:

- по p95 на 3ms (11 vs 14 ms) ~ 27%
- по p99 на 9 ms (23 vs 32 ms) ~ 39%
- при этом потребляя на ~30% меньше heap (195 vs 273 Mb)
- и на ~10% меньше CPU (66% vs 72.8%).

POST /v1/menu-orders — Orders Service

Эксперимент	p95 (ms)	p99 (ms)	Heap	Non-Heap	CPU	Alloc Rate	GC avg (ops/s)	GC pause avg
VT, HttpComponents, parallelism=4	47	147	344 MiB	160 MiB	78.5%	450 MB/s	2.4	3 ms
VT, HttpComponents, parallelism=8	52	141	334 MiB	161 MiB	89.5%	450 MB/s	2.6	3 ms
VT, HttpComponents, parallelism=16	65	225	365 MiB	160 MiB	88%	450 MB/s	2.6	3 ms
VT, HttpComponents, parallelism=8, maxConn=200	62	157	297 MiB	161 MiB	83.5%	450 MB/s	2.7	3.5 ms
VT, JdkClient, parallelism=2	44	109	339 MiB	161 MiB	82.3%	520 MB/s	3.3	4 ms
VT, JdkClient, parallelism=4	38	96	295 MiB	160 MiB	88.5%	520 MB/s	3.1	4 ms
VT, JdkClient, parallelism=6	57	124	310 MiB	160 MiB	98%	520 MB/s	3.1	4 ms
WebFlux, APC=4	48	147	347 MiB	162 MiB	90%	780 MB/s	4.0	3 ms
WebFlux, APC=6	34	122	286 MiB	162 MiB	97%	780 MB/s	4.3	3 ms
WebFlux, APC=8	34	131	345 MiB	163 MiB	98%	780 MB/s	4.1	2 ms
WebFlux, APC=16	43	118	323 MiB	168 MiB	99.5%	780 MB/s	4.1	4 ms

Оптимальная конфигурация Virtual Threads

В конфигурации с `HttpComponentsClientHttpRequestFactory` лучший результат при `-Djdk.virtualThreadScheduler.parallelism=4`:

- p95=47ms
- p99=147ms
- 344 MiB heap
- 78.5% CPU

В конфигурации с `JdkClientHttpRequestFactory` лучший результат при `-Djdk.virtualThreadScheduler.parallelism=4`:

- p95=38ms
- p99=96ms
- 295 MiB heap
- 88.5% CPU

Это на самом деле лучший абсолютный результат по временам ответа среди всех VT-конфигураций для данного эндпоинта. Однако потребление CPU в 88.5% практически не оставляет запаса для всплесков нагрузки и может привести к серьезной деградации сервиса.

Оптимальная конфигурация WebFlux

Лучшие результаты показала конфигурация с `ActiveProcessorCount=6` :

- p95=34ms
- p99=122ms
- 286 MiB heap
- 97% CPU

Сравнение подходов

Для Orders Service картина более нюансированная. По latency виртуальные потоки с `JdkClientHttpRequestFactory` (parallelism=4) опережают лучший WebFlux по:

- p99 на 26ms (96 vs 122ms) ~ 27%
- потреблению CPU на 8.5% (88.5% vs 97%)

Но при этом проигрывает по:

- p95 (34 vs 38ms) ~ 11%
- heap (286 vs 295 MiB) ~ 3%

Самое важное различие WebFlux и виртуальных потоков в данном случае - потребление CPU: WebFlux стабильно загружает процессор на 90–99.5%, оставляя практически нулевой запас. Виртуальные потоки в конфигурации с `HttpComponentsClientHttpRequestFactory` (parallelism=4) при сопоставимой latency потребляют лишь 78.5% CPU, что оставляет ~20% запаса для пиковых нагрузок.

Также у WebFlux значительно выше allocation rate: 780 MB/s против 450–520 MB/s у виртуальных потоков. Одно из возможных объяснений - для эндпоинта с записью в БД эта разница заметнее, вероятнее всего, из-за дополнительных реактивных цепочек для работы с R2DBC.

Общие наблюдения

Наблюдается четкая закономерность: на виртуальных потоках при использовании `HttpComponentsClientHttpRequestFactory` увеличение parallelism от 2 до 8 в большинстве случаев снижает времена ответов, однако дальнейший рост до 16 приводит к их деградации. Это, вероятнее всего, свидетельствует о том, что при `parallelism=16` начинается чрезмерная конкуренция за 2 выделенных CPU.

Также при увеличении `parallelism` происходит снижение потребления heap - при низком `parallelism` в каждый момент времени больше виртуальных потоков находятся в `PARKED` -состоянии (ждут очереди на монтирование), и их стеки живут в heap как `StackChunk` . При высоком `parallelism` больше VT одновременно смонтированы - их стеки на нативных стеках carrier'ов и не занимают heap.

Disclaimer

Очевидно, что результаты эксперимента, проведенного на моей машине будут отличаться от результатов тестирования системы, корректно развернутой в облаке в кластере Kubernetes: будет выше latency из-за сетевых хопов, возможно, будет ниже потребление CPU и heap, что позволит подавать еще большую нагрузку. Кроме того подача нагрузки в течение 10 минут не может служить показателем устойчивости системы к такой нагрузке. Однако в данном случае меня в первую очередь интересовала оценка влияния параметров JVM на поведение системы, процентное соотношение показателей в различных конфигурациях и компромиссы, на которые придется идти при переходе с реактивного стека на императивный. Думаю, что ответы на все эти вопросы я получил.

Практические выводы при использовании виртуальных потоков

- Следует тщательно выбирать оптимальный параметр `jdk.virtualThreadScheduler.parallelism` в контейнерной среде - дефолтное значение (число CPU из cgroups) часто недостаточно для I/O-bound нагрузок.
- Параметры `maxConnectionsPerRoute` и `maxTotalConnections` у `PoolingHttpClientConnectionManager` (в `HttpComponentsClientHttpRequestFactory`) по умолчанию (5 и 25) категорически не подходят для нагрузок 1000+ RPS.
- Выбор `ClientHttpRequestFactory` имеет влияние на производительность сопоставимое с выбором всей модели (WebFlux vs VT).
- На write-heavy эндпоинтах с транзакциями VT оставляют больший запас CPU, что критично для пиковых нагрузок.